

Nexus Pipeline: Build & Deployment Guide

Complete Technical Reference for Autonomous Multi-Model Routing, Obsidian Vault Synchronization, GitHub Auto-Push, and Production Publishing.

1 A.N.T. 3-Layer System Core

OmniRoute & Models

Layer 1

Routes prompts across 420+ intelligence models (Claude 3.7 Sonnet, DeepSeek, Local LLM) based on task latency, cost, and complexity.

Obsidian SSOT Vault

Layer 2

Single Source of Truth at D:\ISHIDA\Projects\ holding blueprints, schemas, architecture logs, and project metadata.

GitHub & Database Hub

Layer 3

Automated Git semantic versioning, secret shielding, schema migrations, and remote repository synchronization.

1. Prompt Intent

User Request / CLI

2. OmniRoute

Model Strategy

3. DB Scaffolding

Prisma / Supabase

4. Code & Build

Antigravity Core

5. Obsidian Sync

D:\ISHIDA\Projects

6. Git & Deploy

GitHub & Cloud

2 Step-by-Step Execution Lifecycle

Phase	Component	Action & Automated Behavior	Output Destination
Step 1: Route	OmniRoute Intelligence	Analyzes prompt complexity and selects optimal model tier (Tier 1: Architect, Tier 2: Coding, Tier 3: Fast scripts).	Memory & Task Dispatcher
Step 2: Schema	Database Hub	Scaffolds schema, connection parameters, ORM models, and migrations if persistence is required.	db/, prisma/, .env
Step 3: Code	Execution Engine	Builds the responsive frontend, robust backend APIs, styling, and validates unit/integration tests.	d:\Z0R0\ Workspace
Step 4: Sync SSOT	Obsidian Bridge	Synchronizes project blueprint, architecture decisions, and task results directly into your knowledge vault.	D:\ISHIDA\Projects\<name>.md
Step 5: Version	GitHub Remote Sync	Shields sensitive files via .gitignore, stages changes, writes semantic commits, and pushes upstream.	GitHub Remote Repository

3 PowerShell & Terminal Control

```
# 1. Inspect pipeline and connected models status .\nexus status # 2. Build a new feature / full application with
automated database and Obsidian sync .\nexus build "Create a real-time responsive analytics dashboard with PostgreSQL"
# 3. Synchronize current workspace state to Obsidian and GitHub remote repository .\nexus sync
```

4 Deployment & Publishing Architecture

The **Trigger Phase** of the B.L.A.S.T. protocol transfers local verified code into production environments automatically or on demand.

● Frontend & Fullstack Web Publishing

- **Firebase Hosting & App Hosting:** Deployed via `npx firebase-tools deploy` for edge caching, Next.js SSR, and SSL.
- **Vercel & Netlify:** Automatic continuous deployment triggered via GitHub webhooks on push to `main` branch.
- **GitHub Pages:** Lightweight static deployment for documentation and client-side single page apps.

● Backend, APIs & Cloud Services

- **Google Cloud Run / GCP:** Containerized microservices with auto-scaling to zero and managed IAM security.
- **Supabase & Postgres Cloud:** Remote database migrations and Edge Function deployments via CLI.
- **Docker / Production Nodes:** Multi-stage build containers ready for Kubernetes or VPS orchestration.

5 Automated CI/CD Workflow Pipeline

Pipeline Stage	Action Performed	Verification & Safety Gate
1. Pre-Commit Linter	Scans workspace for lint errors, unused imports, and formatting defects.	Halts build if fatal syntax or style violations exist.
2. Secret Shielding	Checks <code>.gitignore</code> for <code>.env</code> , private keys, tokens, and <code>.tmp/</code> artifacts.	Prevents accidental credential leakage to public/remote Git.
3. Atomic Git Sync	Formats structured commit: <code>feat(zoro): <intent> [nexus-sync]</code> and pushes upstream.	Ensures commit history remains clean and traceable.
4. Cloud Build Trigger	GitHub Actions or Cloud Build detects new commit and runs build scripts.	Executes end-to-end tests and builds optimized production bundles.
5. Production Release	Deploys release artifact to CDN/Cloud Run with zero-downtime rolling updates.	Health checks confirm 200 OK on live endpoint.

6 System Invariants & Self-Annealing Loop

● Core Invariants

- **Data-First:** Database schemas and JSON payloads must be defined prior to building UI or tools.
- **Ephemeral vs Global:** Local scratch files stay in `.tmp/`; final deliverables sync to GitHub and Obsidian.

● Self-Annealing Recovery

- **OmniRoute Failover:** Falls back automatically to local CLI execution if remote proxy is unavailable.
- **Vault Offline Fallback:** Caches notes in `.tmp/obsidian_cache.json` and auto-retries when vault connects.